

**EXERCISE 3.6**

Not everything works out so nicely. Implement a function, `init`, that returns a `List` consisting of all but the last element of a `List`. So, given `List(1,2,3,4)`, `init` will return `List(1,2,3)`. Why can't this function be implemented in constant time like `tail`?

```
def init[A](l: List[A]): List[A]
```

Because of the structure of a singly linked list, any time we want to replace the tail of a `Cons`, even if it's the last `Cons` in the list, we must copy all the previous `Cons` objects. Writing purely functional data structures that support different operations efficiently is all about finding clever ways to exploit data sharing. We're not going to cover these data structures here; for now, we're content to use the functional data structures others have written. As an example of what's possible, in the Scala standard library there's a purely functional sequence implementation, `Vector` (documentation at <http://mng.bz/Xhl8>), with constant-time random access, updates, `head`, `tail`, `init`, and constant-time additions to either the front or rear of the sequence. See the chapter notes for links to further reading about how to design such data structures.

3.3.2 Improving type inference for higher-order functions

Higher-order functions like `dropWhile` will often be passed anonymous functions. Let's look at a typical example.

Recall the signature of `dropWhile`:

```
def dropWhile[A](l: List[A], f: A => Boolean): List[A]
```

When we call it with an anonymous function for `f`, we have to specify the type of its argument, here named `x`:

```
val xs: List[Int] = List(1,2,3,4,5)
val ex1 = dropWhile(xs, (x: Int) => x < 4)
```

The value of `ex1` is `List(4,5)`.

It's a little unfortunate that we need to state that the type of `x` is `Int`. The first argument to `dropWhile` is a `List[Int]`, so the function in the second argument must accept an `Int`. Scala can infer this fact if we group `dropWhile` into two argument lists:

```
def dropWhile[A](as: List[A])(f: A => Boolean): List[A] =
  as match {
    case Cons(h,t) if f(h) => dropWhile(t)(f)
    case _ => as
  }
```